PROGRAM 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.model_selection import train_test_split

df=pd.read_csv("housing.csv")

#remove missing values
df.dropna(inplace=True)


#create feature avg rooms per household
df["avg_rooms"]=df["total_rooms"]/df["households"]
X=df["avg_rooms"].values
y=df["median_house_value"].values


#data set splitting
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)  #to
prevent the randomising of output

#feature scaling (only for gradient descent)
mean=X_train.mean()
std=X_train.std()
X_train_norm=(X_train-mean)/std
X_test_norm=(X_test-mean)/std
print("mean :  ",mean)
print("std :  ",std)
print('\n')


#initialize parameters
b0=0
b1=0
learning_rate=0.01
iteration=1000  #slope ,b0=0  intercept, b1=0
n=len(X_train_norm)
cost=[ ]  #to store in array
# to count and store values of mse for 1000 iteration)
for i in range(iteration):
        y_pred=b0+b1*X_train_norm #y^=b0+b1x
        mse= np.mean((y_train - y_pred)**2)
        cost.append(mse)
        db0=(-2/n)*np.sum(y_train - y_pred)
```

```
        db1=(-2/n)*np.sum(X_train_norm*(y_train - y_pred))
        b0=b0-learning_rate*db0
        b1=b1-learning_rate*db1

#prediction on test data
y_pred_gd=b0+b1* X_test_norm

plt.scatter(X_test_norm.reshape(-1,1),y_test.reshape(-1,1),color='blue',label='Actual Data')
plt.plot(X_test_norm,y_pred_gd,color='red',label='Regression Line')
plt.xlabel("Average number of rooms")
plt.ylabel("Median house value")
plt.title("Gradient Descent graph")
plt.legend()
plt.show()

#cost convergence graph
plt.plot(range(iteration),cost,color='Green')
plt.xlabel("Average number of rooms")
plt.ylabel("Median house value")
plt.title("Cost Convergence graph")
plt.legend()
plt.grid(True)
plt.show()


#print("Predicted output",y_pred_gd)
print("\nMSE  :  ",mean_squared_error(y_test,y_pred_gd))
print("R2_score :  ",r2_score(y_test,y_pred_gd))

#least square method
model=LinearRegression()
model.fit(X_train.reshape(-1,1),y_train.reshape(-1,1))
y_predict=model.predict(X_test.reshape(-1,1))

MSE = mean_squared_error(y_test,y_predict)
R2 = r2_score(y_test,y_predict)
print("\nMSE  :  ", MSE )
print("R2_score :  ", R2)


print("slope(m) :",model.coef_[0])
print("intercept(c) :",model.intercept_)
print('\n')

m=model.coef_[0]
c=model.intercept_

plt.scatter(X_test.reshape(-1,1),y_test.reshape(-1,1),color='blue',label='Actual Data')
```

```
x_line = np.sort(X_test_norm.reshape(-1,1))
y_line = m*x_line +c
plt.plot(x_line,y_line,color='red',label='Regression Line')
plt.title("Simple linear regresion: Average rooms  VS Median house value")
plt.xlabel("Average number of rooms")
plt.ylabel("Median house value")
plt.legend()
plt.show()
```